

```

# Mount Google Drive for checkpoints
from google.colab import drive
drive.mount('/content/drive')

Mounted at /content/drive

# Step 1: Environment Setup with Version Control
!pip install -Uqq transformers datasets accelerate torchinfo # Tested
versions
!pip install -Uqq torchvision

# Import libraries
import torch
from torch.optim import AdamW
from torch.utils.data import DataLoader
from transformers import (
    DistilBertTokenizerFast,
    DistilBertForSequenceClassification,
    get_linear_schedule_with_warmup
)
from datasets import load_dataset
from accelerate import Accelerator
from tqdm.auto import tqdm

print("Environment setup and imports completed.\n")

# =====
# Configuration
# =====
print("Setting configuration parameters...")

MODEL_NAME = "distilbert-base-uncased"
MAX_LENGTH = 64
BATCH_SIZE = 8
GRAD_ACCUM_STEPS = 2
NUM_EPOCHS = 4
FREEZE_LAYERS = 5
LEARNING_RATE = 2e-5
SAVE_PATH = "/content/drive/MyDrive/emotion_classifier"

print(f"Model: {MODEL_NAME}")
print(f"Max sequence length: {MAX_LENGTH}")
print(f"Batch size: {BATCH_SIZE}")
print(f"Epochs: {NUM_EPOCHS}")
print(f"Learning rate: {LEARNING_RATE}")
print(f"Checkpoint path: {SAVE_PATH}\n")

# =====
# Data Preparation
# =====

```

```

print("Preparing dataset...")

# Load dataset
print("Loading emotion dataset...")
dataset = load_dataset('dair-ai/emotion')

# Initialize tokenizer
print("Initializing tokenizer...")
tokenizer = DistilBertTokenizerFast.from_pretrained(MODEL_NAME)

# Tokenization function
def tokenize_fn(batch):
    return tokenizer(
        batch['text'],
        padding='max_length',
        truncation=True,
        max_length=MAX_LENGTH,
        return_tensors="pt"
    )

# Process dataset
print("Tokenizing dataset...")
dataset = dataset.map(tokenize_fn, batched=True, batch_size=128)
dataset.set_format(type='torch', columns=['input_ids',
    'attention_mask', 'label'])

# Create data loaders
print("Creating data loaders...")
train_loader = DataLoader(dataset['train'], batch_size=BATCH_SIZE,
    shuffle=True)
val_loader = DataLoader(dataset['validation'], batch_size=BATCH_SIZE)
test_loader = DataLoader(dataset['test'], batch_size=BATCH_SIZE)

print(f"Training samples: {len(dataset['train'])}")
print(f"Validation samples: {len(dataset['validation'])}")
print(f"Test samples: {len(dataset['test'])}")
print("Data preparation complete.\n")

# =====
# Model Initialization
# =====
print("Initializing model...")

model = DistilBertForSequenceClassification.from_pretrained(
    MODEL_NAME,
    num_labels=6
)

# Freeze layers
print(f"Freezing first {FREEZE_LAYERS}/6 transformer layers...")

```

```

for layer in model.distilbert.transformer.layer[:FREEZE_LAYERS]:
    for param in layer.parameters():
        param.requires_grad = False

# Verify frozen layers
print("Layer trainability status:")
for i, layer in enumerate(model.distilbert.transformer.layer):
    print(f"Layer {i+1}: {'Trainable' if any(p.requires_grad for p in
layer.parameters()) else 'Frozen'}")
print("Model initialization complete.\n")

# =====
# Training Setup
# =====
print("Setting up training environment...")

accelerator = Accelerator(
    mixed_precision='fp16',
    gradient_accumulation_steps=GRAD_ACCUM_STEPS
)

# Initialize optimizer and scheduler
print("Initializing optimizer and scheduler...")
optimizer = AdamW(
    filter(lambda p: p.requires_grad, model.parameters()),
    lr=LEARNING_RATE
)

total_steps = len(train_loader) // GRAD_ACCUM_STEPS * NUM_EPOCHS
scheduler = get_linear_schedule_with_warmup(
    optimizer,
    num_warmup_steps=0.1 * total_steps,
    num_training_steps=total_steps
)

# Prepare components with Accelerator
print("Preparing components with Accelerator...")
model, optimizer, train_loader, val_loader = accelerator.prepare(
    model, optimizer, train_loader, val_loader
)

print(f"Total training steps: {total_steps}")
print(f"Warmup steps: {0.1 * total_steps:.0f}")
print("Training setup complete.\n")

# =====
# Training Loop
# =====
print("Starting training...")

```

```

for epoch in range(NUM_EPOCHS):
    print("\n" + "="*40)
    print(f"EPOCH {epoch+1}/{NUM_EPOCHS}")
    print("="*40)

    # Training Phase
    print("\nTraining Phase:")
    model.train()
    total_loss = 0
    progress_bar = tqdm(train_loader, desc="Training", leave=False)

    for step, batch in enumerate(progress_bar):
        inputs = {
            'input_ids': batch['input_ids'],
            'attention_mask': batch['attention_mask']
        }
        labels = batch['label']

        outputs = model(**inputs, labels=labels)
        loss = outputs.loss
        total_loss += loss.item()

        accelerator.backward(loss)

        if (step + 1) % GRAD_ACCUM_STEPS == 0:
            optimizer.step()
            scheduler.step()
            optimizer.zero_grad()

        progress_bar.set_postfix(loss=loss.item())

    # Validation Phase
    print("\nValidation Phase:")
    model.eval()
    val_loss = 0
    val_correct = 0

    with torch.no_grad():
        for batch in tqdm(val_loader, desc="Validating", leave=False):
            inputs = {
                'input_ids': batch['input_ids'],
                'attention_mask': batch['attention_mask']
            }
            labels = batch['label']

            outputs = model(**inputs, labels=labels)
            val_loss += outputs.loss.item()
            predictions = torch.argmax(outputs.logits, dim=1)
            val_correct += (predictions == labels).sum().item()

```

```

# Save checkpoint
accelerator.wait_for_everyone()
unwrapped_model = accelerator.unwrap_model(model)
accelerator.save(unwrapped_model.state_dict(),
f"{SAVE_PATH}_epoch{epoch+1}.pt")
print(f"\nCheckpoint saved to {SAVE_PATH}_epoch{epoch+1}.pt")

# Clear GPU cache
torch.cuda.empty_cache()

# Calculate metrics
avg_train_loss = total_loss / len(train_loader)
avg_val_loss = val_loss / len(val_loader)
val_acc = val_correct / len(dataset['validation'])

print(f"\nEpoch {epoch+1} Results:")
print(f"- Training Loss: {avg_train_loss:.4f}")
print(f"- Validation Loss: {avg_val_loss:.4f}")
print(f"- Validation Accuracy: {val_acc:.4f}")

print("\nTraining complete.\n")

# =====
# Final Evaluation
# =====
print("Starting final evaluation...")

model.load_state_dict(torch.load(f"{SAVE_PATH}_epoch{NUM_EPOCHS}.pt"))
model.eval()
test_correct = 0

with torch.no_grad():
    for batch in tqdm(test_loader, desc="Testing"):
        inputs = {
            'input_ids': batch['input_ids'].to(accelerator.device),
            'attention_mask':
batch['attention_mask'].to(accelerator.device)
        }
        labels = batch['label'].to(accelerator.device)

        outputs = model(**inputs)
        predictions = torch.argmax(outputs.logits, dim=1)
        test_correct += (predictions == labels).sum().item()

test_acc = test_correct / len(dataset['test'])
print(f"\nFinal Test Accuracy: {test_acc:.4f}")
print("Evaluation complete.")

Environment setup and imports completed.

```

Setting configuration parameters...

Model: distilbert-base-uncased

Max sequence length: 64

Batch size: 8

Epochs: 4

Learning rate: 2e-05

Checkpoint path: /content/drive/MyDrive/emotion_classifier

Preparing dataset...

Loading emotion dataset...

Initializing tokenizer...

Tokenizing dataset...

```
{"model_id": "a29acb6ce3644bedaa273495d5cb7c45", "version_major": 2, "version_minor": 0}
```

Creating data loaders...

Training samples: 16000

Validation samples: 2000

Test samples: 2000

Data preparation complete.

Initializing model...

Some weights of DistilBertForSequenceClassification were not initialized from the model checkpoint at distilbert-base-uncased and are newly initialized: ['classifier.bias', 'classifier.weight', 'pre_classifier.bias', 'pre_classifier.weight']

You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

Freezing first 5/6 transformer layers...

Layer trainability status:

Layer 1: Frozen

Layer 2: Frozen

Layer 3: Frozen

Layer 4: Frozen

Layer 5: Frozen

Layer 6: Trainable

Model initialization complete.

Setting up training environment...

Initializing optimizer and scheduler...

Preparing components with Accelerator...

Total training steps: 4000

Warmup steps: 400

Training setup complete.

Starting training...

=====

EPOCH 1/4

=====

Training Phase:

```
{"model_id": "6254f4a545ad4502b62df4d072ed86bf", "version_major": 2, "version_minor": 0}
```

Validation Phase:

```
{"model_id": "b2d86623387446709c1351c4c5b6edd0", "version_major": 2, "version_minor": 0}
```

Checkpoint saved to
/content/drive/MyDrive/emotion_classifier_epoch1.pt

Epoch 1 Results:

- Training Loss: 1.0135
- Validation Loss: 0.4360
- Validation Accuracy: 0.8540

=====

EPOCH 2/4

=====

Training Phase:

```
{"model_id": "279dcd8ef22343739d64ec2178389eff", "version_major": 2, "version_minor": 0}
```

Validation Phase:

```
{"model_id": "f98b5916f471402e8eba3cb52e95e8ca", "version_major": 2, "version_minor": 0}
```

Checkpoint saved to
/content/drive/MyDrive/emotion_classifier_epoch2.pt

Epoch 2 Results:

- Training Loss: 0.3589
- Validation Loss: 0.2942
- Validation Accuracy: 0.8945

=====

EPOCH 3/4

=====

Training Phase:

```
{"model_id": "c33eb61884fc4d08821d56adb048520a", "version_major": 2, "version_minor": 0}
```

Validation Phase:

```
{"model_id": "bb1277e9569b476abb45fbc874ff899b", "version_major": 2, "version_minor": 0}
```

Checkpoint saved to
/content/drive/MyDrive/emotion_classifier_epoch3.pt

Epoch 3 Results:

- Training Loss: 0.2578
- Validation Loss: 0.2657
- Validation Accuracy: 0.9040

```
=====
EPOCH 4/4
=====
```

Training Phase:

```
{"model_id": "bbbbdf8c143d4a8e8f73e7ce1cc32820", "version_major": 2, "version_minor": 0}
```

Validation Phase:

```
{"model_id": "0806294f47f840078313dc6f0dbb2e4a", "version_major": 2, "version_minor": 0}
```

Checkpoint saved to
/content/drive/MyDrive/emotion_classifier_epoch4.pt

Epoch 4 Results:

- Training Loss: 0.2157
- Validation Loss: 0.2596
- Validation Accuracy: 0.9090

Training complete.

Starting final evaluation...

```
{"model_id": "da25afb9351d4469a4a2e56c313b3c61", "version_major": 2, "version_minor": 0}
```

Final Test Accuracy: 0.9005
Evaluation complete.

